

Contract postconditions and return type deduction

Date: 2018-10-08
Project: ISO/IEC JTC 1/SC 22/WG 21/C++
Audience subgroup: Core, Evolution
Reply-to: Hubert S.K. Tong <hubert.reinterpretecast@gmail.com>

Issue Description

The interaction between return type deduction and contract postconditions was not raised in the review of the latter feature. Since this introduces the possibility of types and names dependent upon return type deduction, we have a noticeable lack of wording.

Consider:

```
template <typename> struct Tricky { enum { Nested }; };  
template <> struct Tricky<int> { struct Nested { }; };  
auto foo()  
{  
    [[ ensures ret :  
        sizeof(Tricky<decltype(ret)>::Nested) ]] // typename?  
    {  
        return 42;  
    }  
}
```

It has been brought up that the postcondition could not be analyzed in forward declarations in such a situation.

Somewhat separately, recursive calls appearing within the postcondition are currently ill-formed by N4762 subclause 9.11.4.2 [dcl.attr.contract.cond] paragraph 4, which states that the semantic restrictions of a contract condition is the same as if it appeared as the first *expression-statement* in the body of the function.

Possible solutions

Techniques we have for dealing with the apparent parsing issue include using disambiguators in the style of handling dependent names and delayed parsing. Delayed parsing is already necessary for contract conditions of class members declared within the definition of the class since such contract conditions are complete-class contexts. For templates, the return type cannot be deduced until instantiation; therefore, delayed parsing is not sufficient if an abstract syntax tree is to be formed for the template definition (which various implementations rely on in practice).

The following is an enumeration of possible solutions from Richard Smith:

1. Disallow naming the return value in a postcondition if the function has a deduced return type.
2. Allow such naming, but treat the name of the return value as having a dependent type. This means requiring template and typename disambiguators; behavior would be as if the point of instantiation is wherever the definition of the function occurs.
3. Allow such naming as above for templated functions, and for non-templated functions, allow such naming only for definitions. Delaying the parsing of the postcondition until the return type is known is a possible implementation strategy for the non-templated function case of this option.
4. Allow such naming as above, but apply the dependent-type option for non-templated forward declarations.

Notice that a name introduced for the return value associated with a templated entity declared with a deduced return type is type dependent for all options aside from the first (where introducing such a name is ill-formed).

The first option has the benefit of keeping our options open. It will be possible to choose a different solution in the future without breaking or changing the behavior of code that is valid under that option. It is also clearly implementable with clear semantics. As such, this paper provides a proposed resolution (for the consideration of CWG) for the subject issue that implements Option 1.

As to the recursive call prohibition: it could be lifted, but the wording is not presented here. Recursive calls to functions subject to return type deduction are already allowed under 9.1.7.4 [dcl.spec.auto] paragraph 9. Technically, the prohibition could be lifted for preconditions as well as for postconditions.

Proposed resolution for consideration by CWG

The editing instructions within this document use N4762 as its base wording.

☐ Hide inserted text ☐ Hide deleted text

Modify subclause 9.11.4.1 [dcl.attr.contract.syn] paragraph 3:

A *contract-attribute-specifier* using *ensures* is a *postcondition*. It expresses a condition that a function should ensure for the return value and/or the state of objects using a predicate that is intended to hold upon exit from the function. The attribute may be applied to the function type of a function declaration. A postcondition may introduce an identifier to represent the glvalue result or the prvalue result object of the function. The optional *identifier* shall not be present when the declared return type of the function contains a placeholder type (9.1.7.4 [dcl.spec.auto]). [Example:

```
int f(char * c)
    [[ensures res: res > 0 && c != nullptr]];
int g(double * p)
    [[ensures audit res: res != 0 && p != nullptr && *p <= 0.0]];
—end example]
```

Additional discussion points

The first option impacts expressiveness. Should we adopt some form of constrained type specifiers, this option would prohibit applying a postcondition on interfaces expressed with a constrained return type. For example (using the `Concept auto` syntax):

```
Integral auto getWeight() const
    [[ensures res: minWeight <= res && res <= maxWeight]] // error
{
    return 0u;
}
```

The second option introduces complications around the set of declarations available for name lookup in the predicate of affected contracts. The postcondition effectively becomes a template definition, and template-like considerations come into play. For example:

```
bool checkGlobalState(int);
auto foo()
    [[ensures res: checkGlobalState(42u) && checkResult(res)]]; // okay; note: checkGlobalState resolved at "definition"

bool checkResult(int);
auto foo()
    { return 42u; } // okay; checkResult above resolved at "point-of-instantiation" here

bool checkResult(unsigned);

auto foo()
    [[ensures res: checkGlobalState(42u) && checkResult(res)]]; // okay?; checkResult still resolved at "point-of-instantiation"

bool checkGlobalState(unsigned);

auto foo()
    [[ensures res: checkGlobalState(42u) && checkResult(res)]]; // error: checkGlobalState resolved at new "definition" here
```

The third option has a limitation that the first declaration of an affected non-templated function would need to be a definition. It also increases the differences between templated and non-

templated functions; namely, the templated cases may be forward declared. It has the advantage of avoiding the addition of more template-like considerations to the non-template cases.

The fourth option avoids the difference between the templated and non-templated cases of the third option except to admit avoidance of disambiguators where the first declaration of a non-templated function is a definition. It retains the template-like considerations for non-template cases involved in the second option.

Acknowledgements

The author would like to thank Aaron Ballman, Jens Maurer, Richard Smith, and Ville Voutilainen for their feedback on the subject of this paper. As usual, any remaining mistakes are the responsibility of the author.